Aliona Heitz and Deep Shah
CPE 593: Applied Data Structures and Algorithms
Professor Liu
11/6/2026

Milestone 1

Project Topic: Flight Connection Finder

GitHub Repo link: https://github.com/alionaheitz/CPE593-Final-Project/tree/main

Project Overview:

The Flight Connection Finder is a program that helps users find the best route and/or cheapest flight between two airports when a direct flight is not an option. The program allows the user to pick a starting airport and a destination airport, then they have the option to choose: (1) the cheapest flight connection route or (2) the flight with the fewest connecting routes.

The flight information is directly pulled from Google Flights using an API key. It is stored in a hash map, and then it is pulled into a hash map adjacency list, which acts as a graph. Each airport is a key, and its value is a list of the flights leaving that airport. The remaining data structures are a queue, a priority queue (min-heap), and a hash set. Each data structure has a specific role: the hash map instantly looks up the flights leaving any airport, the hash set keeps track of visited airports so no airport is processed twice, the queue explores airports in order of how many flights away they are, and the priority queue always picks the cheapest trip found so far.

While Google Flights is fairly utilized by the public for finding flights, this Flight Connection Finder focuses on the process behind the search, showing how BFS finds the route with the fewest layovers and how Dijkstra's algorithm finds the cheapest route across a network of airports.

Project Planning:

Phase 1: Building the Repository ***(Complete)***

- Create the repository and give access to all users
- Created all directories (finalProject, docs, code)

Phase 2: Research Dataset: Google Flights API Key ***(Complete)***

- The Google Flights data includes the airports, booking options, flight results, and price insights.
- https://serpapi.com/google-flights-api

Phase 3: Web Scraping and Hash Map Construction

- Pulling data from the Google Flights API Key
- Define classes for Airport and Flight Information in the hash map
- Get the data and embed it into our hash map

Phase 4: Graph Construction

- Create the hash map adjacency list (graph) based on the hash map created above.

Phase 5: Breadth-First Search Algorithm Implementation

- Create a queue and hash sets
    - Queue: Visits the nodes level by level. (The queue ensures the algorithm explores the sequence of the graph in a specific order)
    - Hash Set: Memory Tracker for airports visited (Has every one that has been visited)
- Implement the BFS algorithm using the queue and hash set to find the fewest layovers

Phase 6: Dijkstra's Algorithm Implementation

- Create a minimum heap (heapq)
- Implement Dijkstra algorithm with a priority queue (heapq) and hash set to find the cheapest route

Phase 7: User Menu and Route Output

- Create a user input menu where the user enters a starting airport, a destination airport, and chooses option (1) cheapest route or (2) fewest flights
- Add an option to show both results side by side, so the user can compare the cheapest route with the fewest-flights route.

Phase 8: Testing, Handling Edge Cases, and Documentation

- Test both algorithms on a small set of flights where the correct answers can be worked out by hand.
- Handle edge cases like no available route, the same starting and destination airports, and invalid airport codes.
- Complete the README.md with the project description and instructions for running the code.
- Write the final paper in IEEE format explaining the data structures, algorithms, testing done, and their time complexity.

Workload/Tasks Assigned:

- Phase 1: Building repo (Aliona)
- Phase 2: Data Research (Deep)
- Phase 3: Web scraping (Deep), Hash map construction (Deep and Aliona)
- Phase 4: Graph construction (Aliona)
- Phase 5: BFS algorithm (Deep)
- Phase 6: Dijkstra algorithm (Aliona)
- Phase 7: User Menu (Aliona)
- Phase 8: Testing and handling edge cases (Deep), Documentation (Deep and Aliona)